

Technical Design Doc: Evolve for Reflex

Author: Janvi Palan

Last updated: 2026-08-04

Status: Draft | In Review | Approved

Reviewers: Chao Wang Dafang He Tim Chu Gideon Kim (He Him) James Li J.J Hu
Jacob Gao Dinesh Govindaraj Helen Xu

Related: Detect Evaluation Proposal (Chao Wang)

1. Overview

In the current reflex workflow, playbook(s) are typically run by the PM agent to ideate on hypothesis grounded, clear and actionable. These playbooks are living artifacts, a series of prompts and steps and references to knowledge bases that need both institutional knowledge as well as analytical grounding to generate the hypotheses. While the playbooks are being iterated on, manually or by reflex, there is no clear notion of measuring regression and learning from the regressions to prevent the quality of hypothesis from deteriorating.

The Evolution Agent closes that loop - it claude-replays recorded playbook runs against a candidate playbook(s), scores the create hypothesis card with the evaluation framework and human-calibrated LLM judge, reflects on the failures in natural language, proposes a **section-level** playbook(s) edit, and — only if the edit is Pareto-better — writes it to **evolve/proposed/** for **human approval**. More information about the eval phase for detect can be found here: [\[living doc\] Evaluation Proposal -- Reflex Detect](#) .

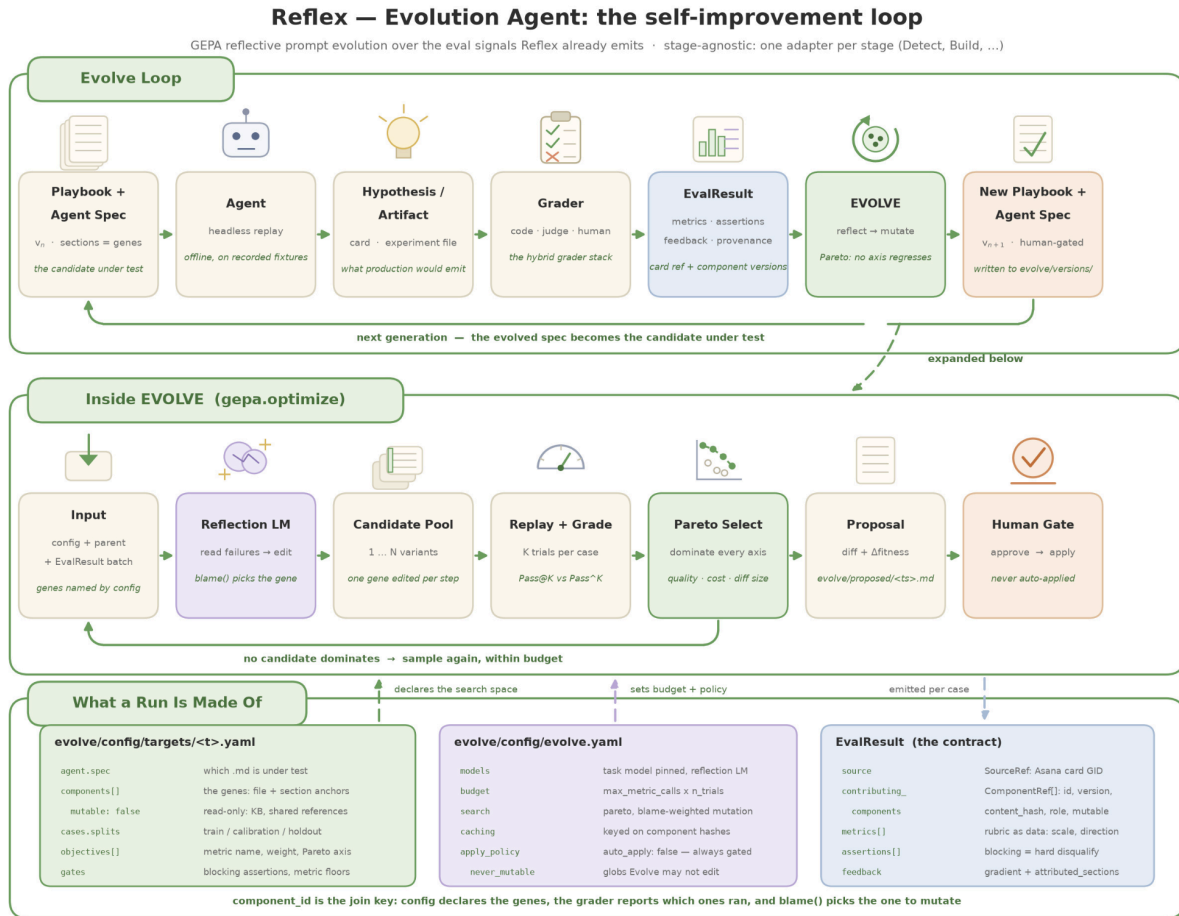
2. Evaluation Methodology

Reflex has various stages, and each can be built with its own evaluation methodology. Evaluation methodology for some of the existing stages is captured in table 1.

Eval signal	Component	Where it lives today	GEPA role
Composite score (1–5)	Detect Stage	HypothesisJudgeV erdict.composite _quality, ProcessJudgeVerd ict.overall_proc	scalar fitness (a Pareto axis)

Eval signal	Component	Where it lives today	GEPA role
		<code>ess_quality</code> , Build <code>pass_rate</code>	
Binary score	Build Stage (Ads Build Agents)	Build tier1/2/3 per-test <code>passed</code> ; Judge <code>overall_verdict</code> thresholded	scalar fitness (a Pareto axis)
Rationale feedback	Detect Stage/Build stage	Judge per-dimension <code>*_rationale</code> , <code>strengths</code> , <code>improvements</code> , <code>missing_checks</code> ; Build <code>error_message</code> / "Key Findings"	textual feedback (ASI) — the reflective "gradient"

3. Evolution Agent : The self-improvement loop



3.1 Scope

In	Out
Rewriting sections of agent specs (<code>detect/agents/pm_agent.md</code>)	Adding or removing playbooks — the PM Agent owns the roster
Rewriting sections of playbooks	Editing the knowledge base — the Feedback Curator owns it
Detect and Build in v1	Simulate, Prove, Router — one adapter each, later
Offline ranking against recorded fixtures	Live-run optimization

3.2 EvalResult

`evolve/infra/schemas/eval_result.py` is the only coupling point between a particular stage's evals and Evolve. Each `MetricScore` carries its own name, scale bounds, direction, and weight, which puts a stage's rubric in the values Evolve reads rather than in Evolve's code.

Python

```
class MetricScore(BaseModel):
    name: str          # 'rigor' or 'tier1_pass_rate' - the stage names it
    value: float
    scale_min: float = 0.0
    scale_max: float = 1.0
    direction: Direction = "maximize"
    weight: float = 0.0    # 0 = excluded from the composite
    is_pareto_axis: bool = True
    rationale: str | None  # feeds the textual gradient

    @property
    def normalized(self) -> float:
        """Value mapped to [0,1], higher always better, for Pareto comparison."""
```

3.3 Methodology

3.3.1 Playbook + Agent Spec (v□)

The candidate under test is a markdown file — a playbook, or a PM or build agent spec. Its heading-anchored sections are the mutable components. `components[ ].sections` in the target YAML declares which sections enter the search space.

3.3.2 Agent

A headless Claude Code run of that spec against recorded fixtures. Replay is offline and deterministic: fixtures are snapshotted Asana, Presto, and MCP responses, so a gepa rollout never touches live services. The task model and temperature are pinned, so we keep score deltas attributable to the component edit rather than to sampling.

3.3.3 Hypothesis / Artifact

The output that is emitted — an opportunity card for Detect, a PR for Build.

3.3.4 Grader/Evaluator

Every grader type produces both a number and a rationale, which is then consumed by GEPA.

3.3.5 EvalResult

This is the sole coupling point between the eval side and Evolve. It is an API contract that carries the scores, the textual feedback that serves as the reflective gradient, grader provenance, and the trial count. `scores` is an open dict, so the eval side adds or renames a metric without an Evolve code change and a new metric becomes a Pareto axis automatically. Evolve asserts the major `contract_version` at startup, so a breaking change fails in seconds.

3.3.6 EVOLVE

`gepa.optimize`. Reflect on the failures, mutate one component, keep a candidate only if it Pareto-dominates its parent. Expanded in section 2.

3.3.6.1 Input

The target config, the parent candidate, and a batch of `EvalResults`. Mutable components are named by config.

3.3.6.2 Reflection LM

Reads the failing cases and their rationales and proposes an edit. `blame()` selects which component to mutate from the ones the evals reported as contributing.

3.3.6.3 Candidate Pool

N variants of the input/components, each differing from the parent in exactly one component. Editing one component per step keeps a proposal small enough to score cleanly and to review against a diff.

3.3.6.4 Replay + Grade

K trials per case, because replay is stochastic. Pass@K (at least 1 of K succeeds) is the optimistic axis while Pass^K (all K succeed) is the reliability axis and the Build optimization target. An edit that raises Pass@K but not Pass^K made the agent luckier but not better.

3.3.6.5 Pareto Select

A candidate survives only if it is **no worse** on every objective and strictly better on **at least one**.

Adapter	Objectives (direction)
Build	eval pass-rate (max); BuildValidator success (max); diff size (min, tie-break)
Detect	composite hypothesis quality (max); PM precision (max); process quality (max); dead-end hits (min)

3.3.6.6 Proposal

`evolve/proposed/<ts>.md`, carrying the before/after diff, the reflection that motivated the edit, and the fitness delta on every objective.

3.3.6.7 Human Gate

No edit auto-applies. The config loader rejects a run that sets `auto_apply: true`, so the gate cannot be bypassed by configuration. If no candidate dominates the parent, the loop samples again until `max_metric_calls` is exhausted.

3.3.7 Playbook + Agent Spec (v_{N+1})

Written to `evolve/versions/<agent>/vN.md` after a human approves, which doubles as the prompt version history Reflex currently lacks. The evolved spec becomes the next generation's candidate under test.

4. Running Evolve

It can be triggered with `run_evolution` - which runs one target.

`evolve/infra/run_evolution.py` It takes a target ID and no arguments that change what the run means — components, cases, objectives, and budget all come from the YAML, so the command line carries no design decisions.

4.1 API

None

```
# Resolve the config and print the run without spending budget.
python3 evolve/infra/run_evolution.py --target detect_pm_agent
--dry-run
```

```
# Search: GEPA over the train split, ending in a proposal under
evolve/proposed/.
```

```
python3 evolve/infra/run_evolution.py --target detect_pm_agent

# Every target with `enabled: true`, sequentially.
python3 evolve/infra/run_evolution.py --all-enabled

# Approval-time re-run of the whole case bank against a proposed
candidate.
python3 evolve/infra/run_evolution.py --target detect_pm_agent
--candidate evolve/proposed/20260807-pm-agent-phase-2.md
--full-rerun
```

Startup runs checks before the first rollout, so a run that cannot finish dies in seconds: `assert_contract_compatible()` compares the stage's emitted contract against `contract_version`, the module named by `target.adapter` must import, and `cases.index` must exist with at least `min_train_cases` in the train split. Two run kinds share the entry point:

Kind	Split	Ends in	Trigger
Search	<code>train</code> , then <code>validation</code> on the surviving candidate	A proposal in <code>evolve/proposed/</code>	A person decides to evolve a target
Landing re-run	Every case in the bank	Pass or fail against <code>require_full_rerun_before_landing</code>	A person approves a proposal

Both append a record to `evolve/state/evolution_log.jsonl` under one `run_id`, which is also the rollout-cache namespace, so a re-run of the same target reuses cached rollouts for genes it did not mutate.

Runs are manual in v1. A person invokes the command, typically after a Detect rotation closes or a Build eval regresses. Scheduling is deferred because a run costs up to 450 headless invocations and produces a proposal a human has to read, so a cadence would queue proposals faster than they get reviewed. Two triggers are worth wiring once review throughput is known: the PM Agent's Phase 6 rotation audit completing, for Detect targets, and a tier1 regression landing in `build/state/eval_reports/`, for Build targets. Either one would call the same CLI, since neither changes what a run is.

4.2 What a run is made of

File	Role
<code>evolve/config/targets/<t>.yaml</code>	Declares the search space: which spec is under test, which sections are mutable, case splits, objectives and their Pareto axes, blocking gates
<code>evolve/config/evolve.yaml</code>	Budget and policy: pinned task and reflection models, <code>max_metric_calls × n_trials</code> , search strategy, cache keying, <code>never_mutable</code> globs
<code>EvalResult</code>	Emitted per case by the stage adapter — the only object Evolve consumes

5. Success Criteria

Evolve optimizes other agents - these are the signals that tell us Evolve is worth running. All are read off `evolve/state/evolution_log.jsonl` and the proposal records, so none needs new instrumentation.

Signal	Definition	Reads as healthy	Reads as broken
Proposal acceptance rate	approved proposals / proposals surfaced	30–70%	<10% (Evolve wastes review); >90% (human gate rubber-stamping)
Held-out regression catch	accepted edits caught by the landing re-run before landing / accepted edits that regress the held-out slice	catches every regressing edit	a landed edit later regresses the held-out slice (section 4.1)
Landed-edit durability	landed edits not reverted or superseded within two rotations / landed edits	most persist	churn — edits landed then undone

Signal	Definition	Reads as healthy	Reads as broken
Net fitness delta	aggregate Pareto-axis improvement of landed edits vs the pre-Evolve baseline spec	positive on the target axes	flat or negative after review cost
Review cost	median human minutes per surfaced proposal	bounded enough to keep pace with the run cadence (section 9.3)	proposals queue faster than they are read